Assignment 4

Name: Ridhi **Roll No:** 1024030528

Problem 1: Course Registration Portal

Design a comprehensive course registration portal combining multiple collection types for optimal data handling, prerequisites tracking, and waitlist management. A HashMap<String, Course> is used for fast course lookups by course code. Each Course contains a HashSet of unique enrolled student IDs, an ArrayList for ordered prerequisite codes, and a LinkedList acting as a first-come, first-served waitlist queue. Registration requests are handled with capacity checks and automated waitlist promotion logic.

Source Code (CourseRegistrationPortal.java)

```
/*
 * Assignment 4 - Problem 1: Course Registration Portal
 * Name    : Ridhi
 * Roll No : 1024030528
 */
import java.util.*;

class Course {
    private final String code;
    private final String name;
    private final int capacity;
    private final Set<String> enrolled = new HashSet<>();            // unique enrolled student IDs
    private final List<String> prerequisites = new ArrayList<>();    // ordered prerequisite codes
    private final LinkedList<String> waitlist = new LinkedList<>();  // FCFS waitlist queue

    Course(String code, String name, int capacity, String... prereqs) {
        this.code = code;
        this.name = name;
        this.capacity = capacity;
        prerequisites.addAll(Arrays.asList(prereqs));
    }

    String getCode() { return code; }
    String getName() { return name; }
    int getCapacity() { return capacity; }
    Set<String> getEnrolled() { return enrolled; }
    List<String> getPrerequisites() { return prerequisites; }
    LinkedList<String> getWaitlist() { return waitlist; }
    boolean isFull() { return enrolled.size() >= capacity; }

    @Override
    public String toString() {
        return code + " (" + name + ") enrolled=" + enrolled.size() + "/" + capacity
                + ", prereqs=" + prerequisites + ", waitlist=" + waitlist;
    }
}

public class CourseRegistrationPortal {
    private final Map<String, Course> courses = new HashMap<>();             // fast lookup by course code
    private final Map<String, Set<String>> completed = new HashMap<>();      // student -> completed courses

    public void addCourse(Course c) { courses.put(c.getCode(), c); }

    public void markCompleted(String studentId, String courseCode) {
        completed.computeIfAbsent(studentId, k -> new HashSet<>()).add(courseCode);
    }

    /** Handles a registration request with prerequisite + capacity checks. */
    public String register(String studentId, String courseCode) {
        Course c = courses.get(courseCode);
        if (c == null) return "FAILED: Course " + courseCode + " does not exist.";

        if (c.getEnrolled().contains(studentId))
            return "FAILED: " + studentId + " is already enrolled in " + courseCode + ".";
        if (c.getWaitlist().contains(studentId))
            return "FAILED: " + studentId + " is already on the waitlist for " + courseCode + ".";
        Set<String> done = completed.getOrDefault(studentId, Collections.emptySet());
```

```
        for (String pre : c.getPrerequisites()) {
            if (!done.contains(pre))
                return "FAILED: " + studentId + " has not completed prerequisite " + pre + ".";
        }

        if (!c.isFull()) {
            c.getEnrolled().add(studentId);
            return "SUCCESS: " + studentId + " enrolled in " + courseCode + ".";
        }
        c.getWaitlist().addLast(studentId);
        return "WAITLISTED: " + courseCode + " is full. " + studentId
                + " is at position " + c.getWaitlist().size() + " on the waitlist.";
    }

    /** Drops a student and automatically promotes the first waitlisted student. */
    public String drop(String studentId, String courseCode) {
        Course c = courses.get(courseCode);
        if (c == null) return "FAILED: Course " + courseCode + " does not exist.";

        if (c.getWaitlist().remove(studentId))
            return "SUCCESS: " + studentId + " removed from waitlist of " + courseCode + ".";

        if (!c.getEnrolled().remove(studentId))
            return "FAILED: " + studentId + " is not registered in " + courseCode + ".";

        StringBuilder msg = new StringBuilder("SUCCESS: " + studentId + " dropped " + courseCode + ".");
        promoteFromWaitlist(c, msg);
        return msg.toString();
    }

    /** Automated waitlist promotion: first-come, first-served. */
    private void promoteFromWaitlist(Course c, StringBuilder msg) {
        while (!c.isFull() && !c.getWaitlist().isEmpty()) {
            String next = c.getWaitlist().pollFirst();
            c.getEnrolled().add(next);
            msg.append(" PROMOTED: ").append(next).append(" from waitlist.");
        }
    }

    public void printCourse(String code) { System.out.println(courses.get(code)); }

    public static void main(String[] args) {
        CourseRegistrationPortal portal = new CourseRegistrationPortal();
        portal.addCourse(new Course("CS101", "Intro to Programming", 2));
        portal.addCourse(new Course("CS201", "Data Structures", 2, "CS101"));
        portal.addCourse(new Course("CS301", "Algorithms", 1, "CS101", "CS201"));

        for (String s : new String[]{"S1", "S2", "S3", "S4"}) portal.markCompleted(s, "CS101");
        portal.markCompleted("S1", "CS201");

        System.out.println("--- Registrations ---");
        System.out.println(portal.register("S1", "CS101"));
        System.out.println(portal.register("S2", "CS101"));
        System.out.println(portal.register("S3", "CS101"));   // waitlisted
        System.out.println(portal.register("S4", "CS101"));   // waitlisted
        System.out.println(portal.register("S1", "CS101"));   // duplicate
        System.out.println(portal.register("S5", "CS201"));   // missing prerequisite
        System.out.println(portal.register("S1", "CS301"));   // S1 has both prerequisites -> success
        System.out.println(portal.register("S9", "CS999"));   // invalid course
        portal.printCourse("CS101");

        System.out.println("\n--- Drops & Promotion ---");
        System.out.println(portal.drop("S1", "CS101"));       // S3 promoted
        System.out.println(portal.drop("S4", "CS101"));       // removed from waitlist
        System.out.println(portal.drop("S7", "CS101"));       // not registered
        portal.printCourse("CS101");
    }
}
```

Output

Compiled with `javac CourseRegistrationPortal.java` and run with `java CourseRegistrationPortal`:

```
$ java CourseRegistrationPortal
--- Registrations ---
SUCCESS: S1 enrolled in CS101.
SUCCESS: S2 enrolled in CS101.
WAITLISTED: CS101 is full. S3 is at position 1 on the waitlist.
WAITLISTED: CS101 is full. S4 is at position 2 on the waitlist.
FAILED: S1 is already enrolled in CS101.
FAILED: S5 has not completed prerequisite CS101.
SUCCESS: S1 enrolled in CS301.
FAILED: Course CS999 does not exist.
CS101 (Intro to Programming) enrolled=2/2, prereqs=[], waitlist=[S3, S4]

--- Drops & Promotion ---
SUCCESS: S1 dropped CS101. PROMOTED: S3 from waitlist.
SUCCESS: S4 removed from waitlist of CS101.
FAILED: S7 is not registered in CS101.
CS101 (Intro to Programming) enrolled=2/2, prereqs=[], waitlist=[]
```

Problem 2: Media Player Queue Engine

Build a media player queue engine that supports fast insertions at arbitrary positions, recent history tracking, and bidirectional navigation. A LinkedList<Song> manages playback queues, with features to add songs to the front (priority queueing), remove duplicate song entries efficiently, and use a ListIterator to traverse the playlist both forward and backward without index-based performance bottlenecks.

Source Code (MediaPlayerQueue.java)

```
/*
 * Assignment 4 - Problem 2: Media Player Queue Engine
 * Name    : Ridhi
 * Roll No : 1024030528
 */
import java.util.*;

class Song {
    private final String title;
    private final String artist;

    Song(String title, String artist) { this.title = title; this.artist = artist; }
    String getTitle() { return title; }
    String getArtist() { return artist; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Song)) return false;
        Song s = (Song) o;
        return title.equalsIgnoreCase(s.title) && artist.equalsIgnoreCase(s.artist);
    }

    @Override
    public int hashCode() { return Objects.hash(title.toLowerCase(), artist.toLowerCase()); }

    @Override
    public String toString() { return "\"" + title + "\" - " + artist; }
}

public class MediaPlayerQueue {
    private final LinkedList<Song> playlist = new LinkedList<>();
    private final Deque<Song> history = new ArrayDeque<>();   // recent history (most recent first)
    private static final int HISTORY_LIMIT = 5;

    private ListIterator<Song> it;      // bidirectional cursor, no index-based access
    private boolean lastMovedForward;   // tracks direction to avoid repeating a song when direction flips
    private Song current;

    public MediaPlayerQueue() { resetCursor(); }

    private void resetCursor() { it = playlist.listIterator(); current = null; lastMovedForward = true; }

    /** Normal add to the end of the queue. */
    public void addSong(Song s) { playlist.addLast(s); resetCursor(); }

    /** Priority queueing: add to the front. O(1). */
    public void addToFront(Song s) { playlist.addFirst(s); resetCursor(); }

    /** Insert at an arbitrary position using a ListIterator (no repeated get(i) calls). */
    public void insertAt(int position, Song s) {
        if (position < 0 || position > playlist.size())
            throw new IndexOutOfBoundsException("Invalid position: " + position);
        ListIterator<Song> li = playlist.listIterator(position);
        li.add(s);
        resetCursor();
    }

    /** Removes duplicate entries in one pass using a HashSet + iterator.remove(). O(n). */
    public int removeDuplicates() {
        Set<Song> seen = new HashSet<>();
        int removed = 0;
        Iterator<Song> iter = playlist.iterator();
        while (iter.hasNext()) {
            if (!seen.add(iter.next())) { iter.remove(); removed++; }
        }
```

```
        resetCursor();
        return removed;
    }

    /** Move forward using ListIterator. */
    public Song playNext() {
        if (!lastMovedForward && it.hasNext()) it.next();   // skip the element returned by previous()
        if (!it.hasNext()) return null;
        current = it.next();
        lastMovedForward = true;
        recordHistory(current);
        return current;
    }

    /** Move backward using ListIterator. */
    public Song playPrevious() {
        if (lastMovedForward && it.hasPrevious()) it.previous();  // skip the element returned by next()
        if (!it.hasPrevious()) return null;
        current = it.previous();
        lastMovedForward = false;
        recordHistory(current);
        return current;
    }

    private void recordHistory(Song s) {
        history.remove(s);
        history.addFirst(s);
        if (history.size() > HISTORY_LIMIT) history.removeLast();
    }

    public void showPlaylist() {
        System.out.println("Playlist: " + playlist);
    }

    public void showHistory() {
        System.out.println("Recent history: " + history);
    }

    public static void main(String[] args) {
        MediaPlayerQueue player = new MediaPlayerQueue();
        player.addSong(new Song("Believer", "Imagine Dragons"));
        player.addSong(new Song("Shape of You", "Ed Sheeran"));
        player.addSong(new Song("Believer", "Imagine Dragons"));   // duplicate
        player.addSong(new Song("Faded", "Alan Walker"));
        player.addSong(new Song("Shape of You", "Ed Sheeran"));    // duplicate
        player.showPlaylist();

        System.out.println("\n--- Priority queueing (add to front) ---");
        player.addToFront(new Song("Blinding Lights", "The Weeknd"));
        player.showPlaylist();

        System.out.println("\n--- Insert at position 2 ---");
        player.insertAt(2, new Song("Senorita", "Shawn Mendes"));
        player.showPlaylist();

        System.out.println("\n--- Remove duplicates ---");
        System.out.println("Duplicates removed: " + player.removeDuplicates());
        player.showPlaylist();

        System.out.println("\n--- Forward navigation ---");
        for (int i = 0; i < 4; i++) System.out.println("Now playing: " + player.playNext());

        System.out.println("\n--- Backward navigation ---");
        for (int i = 0; i < 3; i++) System.out.println("Now playing: " + player.playPrevious());

        System.out.println();
        player.showHistory();
    }
}
```

Output

Compiled with `javac MediaPlayerQueue.java` and run with `java MediaPlayerQueue`:

```
$ java MediaPlayerQueue
Playlist: ["Believer" - Imagine Dragons, "Shape of You" - Ed Sheeran, "Believer" - Imagine Dragons,
    "Faded" - Alan Walker, "Shape of You" - Ed Sheeran]

--- Priority queueing (add to front) ---
Playlist: ["Blinding Lights" - The Weeknd, "Believer" - Imagine Dragons, "Shape of You" - Ed Sheeran,
    "Believer" - Imagine Dragons, "Faded" - Alan Walker, "Shape of You" - Ed Sheeran]

--- Insert at position 2 ---
Playlist: ["Blinding Lights" - The Weeknd, "Believer" - Imagine Dragons, "Senorita" - Shawn Mendes,
    "Shape of You" - Ed Sheeran, "Believer" - Imagine Dragons, "Faded" - Alan Walker, "Shape of You" - Ed
     Sheeran]

--- Remove duplicates ---
Duplicates removed: 2
Playlist: ["Blinding Lights" - The Weeknd, "Believer" - Imagine Dragons, "Senorita" - Shawn Mendes,
    "Shape of You" - Ed Sheeran, "Faded" - Alan Walker]

--- Forward navigation ---
Now playing: "Blinding Lights" - The Weeknd
Now playing: "Believer" - Imagine Dragons
Now playing: "Senorita" - Shawn Mendes
Now playing: "Shape of You" - Ed Sheeran

--- Backward navigation ---
Now playing: "Senorita" - Shawn Mendes
Now playing: "Believer" - Imagine Dragons
Now playing: "Blinding Lights" - The Weeknd

Recent history: ["Blinding Lights" - The Weeknd, "Believer" - Imagine Dragons, "Senorita" - Shawn Mendes,
     "Shape of You" - Ed Sheeran]
```